《算法分析与设计B》

贪心算法
Greedy Algorithm

计算机与人工智能学院
李晓航
xhli_scce@swjtu.edu.cn

Last: 2026-06

一、贪心算法概述

- 1.1 贪心算法的基本概念
- 1.2 贪心算法基本要素
- 1.3 贪心算法求解过程
- 1.4 贪心算法引例>>程序存储问题实例

1.1 贪心算法基本概念

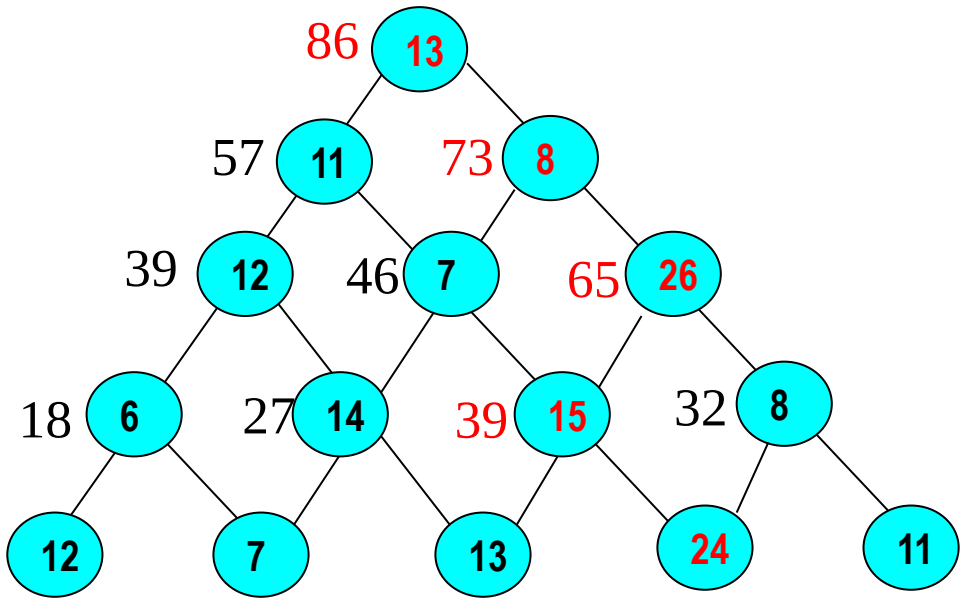
- 贪心算法也叫贪婪算法，其在解决问题时总是作出在当前看来最好的选择。也就是说贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的局部最优选择。
- 由于贪心算法每次都是针对局部最优做选择，且一旦做出选择就不会再改变，因此其不能对所有问题都得到整体最优解。
- 不过对许多满足贪心选择性质的问题，如任务安排问题、单源最短路径问题、最小生成树问题等，贪心算法能高效的产生整体最优解。
- 此外在一些情况下，即使贪心算法不能得到整体最优解，其最终结果也是近似最优解。

3

贪心算法基本概念

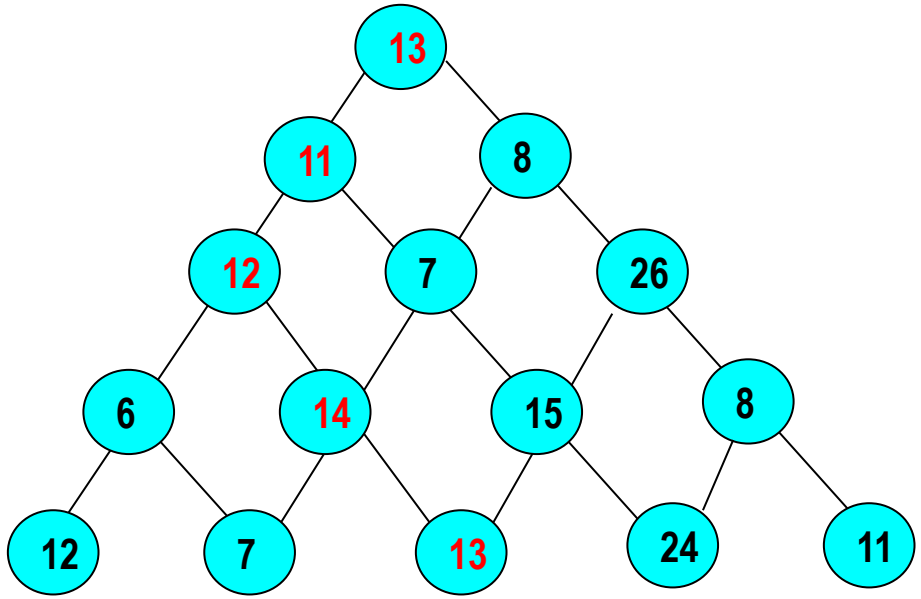
贪心算法不能保证对所有问题都得到整体最优解，为什么？

以数塔问题为例：



动态规划法

86：24-15-26-8-13



贪心法

63：13-11-12-14-13

1.2 贪心算法基本要素

[贪心算法] 具备贪心选择和最优子结构性质的最优化问题。

整体的最优解可通过一系列局部最优解达到。每次的选择可以依赖以前作出的选择，但不能依赖于后面的选择。

问题的整体最优解中包含着它的子问题的最优解.

所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

贪心算法通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

1.3 贪心算法求解过程

用贪心法求解问题应该考虑如下几个方面：

（1）候选集合C：为了构造问题的解决方案，有一个候选集合 C 作为问题的可能解，即问题的最终解均取自于候选集合C。例如，在付款问题中，各种面值的货币构成候选集合。

（2）解集合S：随着贪心选择的进行，解集合 S 不断扩展，直到构成一个满足问题的完整解。例如，在付款问题中，已付出的货币构成解集合。

（3）解决函数solution：检查解集合 S 是否构成问题的完整解。例如，在付款问题中，解决函数是已付出的货币金额恰好等于应付款。

（4）选择函数select：即**贪心策略，这是贪心法的关键**，它指出哪个候选对象最有希望构成问题的解（在候选对象里挑一个当前最优）。选择函数通常和目标函数有关（即应该根据目标函数确定针对候选对象的哪个属性做贪心选择）。

（5）可行函数feasible：检查解集合中加入一个候选对象是否可行，即解集合扩展后是否满足约束条件。例如，在付款问题中，可行函数是每一步选择的货币和已付出的货币相加不超过应付款。

贪心算法求解过程

```
Greedy(C)  //C是问题的输入集合即候选集合
{
    S={ };  //初始解集合为空集
    while (not solution(S))  //集合S没有构成问题的一个解
    {
       x=select(C);    //在候选集合C中做“贪心”选择
       if feasible(S, x)  //判断集合 S 中加入 x 后的解是否可行
      {
          S=S+{x};
          C=C-{x};
       }
    }
    return S;
}
```

将问题的求解过程看作一系列选择，每次选择都是当前状态下的最好选择(局部最优解)。每作一次选择后，所求问题会简化为一个规模更小的子问题，从而通过每一步的最优解逐步达到整体最优解。

1.4 贪心算法引例>>程序存储问题实例

- 问题描述：

  设有 n 个程序 {1,2,…, n } 要存放在长度为 L 的磁带上。程序 i 存放在磁带上的长度是 li，1 ≤i ≤ n。

  程序存储问题要求确定这 n 个程序在磁带上的一个存储方案，使得能够在磁带上存储尽可能多的程序。

- 算法任务：

  对于给定的 n 个程序存放在磁带上的长度，编程计算磁带上最多可以存储的程序数。

贪心算法引例>>程序存储问题实例

- 数据输入：

  第一行是 2 个正整数，分别表示文件个数 n 和磁带的长度 L。接下来的1 行中，有 n 个正整数，表示程序存放在磁带上的长度。

- 结果输出:

  最多可以存储的程序数。

- 输入示例

  6 50

  2 3 13 8 80 20

- 输出示例

  5

i	0	1	2	3	4	5
x	2	3	13	8	80	20

贪心算法引例>>程序存储问题实例

```
int greedy( vector<int> x, int m){
        int i=0, sum=0, n=x.size();
        sort(x.begin(),x.end());
        while(i < n){
                sum += x[i];
                if(sum <= m) i++;
                else return i;
        }
        return n;   //所有的程序都没有磁带长
}
```

为什么要排序？

i	0	1	2	3	4	5
x	2	3	8	13	20	80

二、活动安排问题

- 问题描述
  - 设有 n 个活动的集合 E = {1、2、3、…、n}，其中每个活动都要求使用同一资源，而在同一时间内只有一个活动能使用这一资源。
  - 每个活动 i 都有一个要求使用该资源的起始时间 s_i 和一个结束时间 f_i（$s_i < f_i$）。如果选择了活动 i，则它在半开区间 $[s_i, f_i)$内占用资源。
  - 若区间 [si, fi)与区间 [sj,fj,) 不相交，则称活动 i 与活动 j 是相容的。
  - 活动安排问题就是要在所给的活动集合中选出最大的相容活动子集。
- 活动安排问题要求高效地安排一系列争用某一公共资源的活动
  - 贪心算法提供了一个简单、漂亮的方法使得尽可能多的活动能兼容地使用公共资源。

活动安排问题的贪心策略

- 活动安排问题的关键是如何按照一定的顺序安排活动，使得选出的活动间相容并能安排尽量多的活动。
- 一种贪心策略是取当前活动集合中“结束时间”最早的相容活动，这样可以为未安排的活动留下尽可能多的时间。
  - 这种贪心选择的目的是使剩余时间段极大化。
  - 为了在每一次贪心选择时快速查找具有最早结束时间的相容活动，可以先把 n 个活动按结束时间非减序排列。
- 贪心算法并不总能求得问题的整体最优解，但对于活动的安排问题，按结束时间先后选择相容活动，总能求得整体最优解。
  - 这个结论可以用数学归纳法证明。
  - 思考：如果优先取“开始时间”最早，或“活动时长最短”，是否能得到最优解？

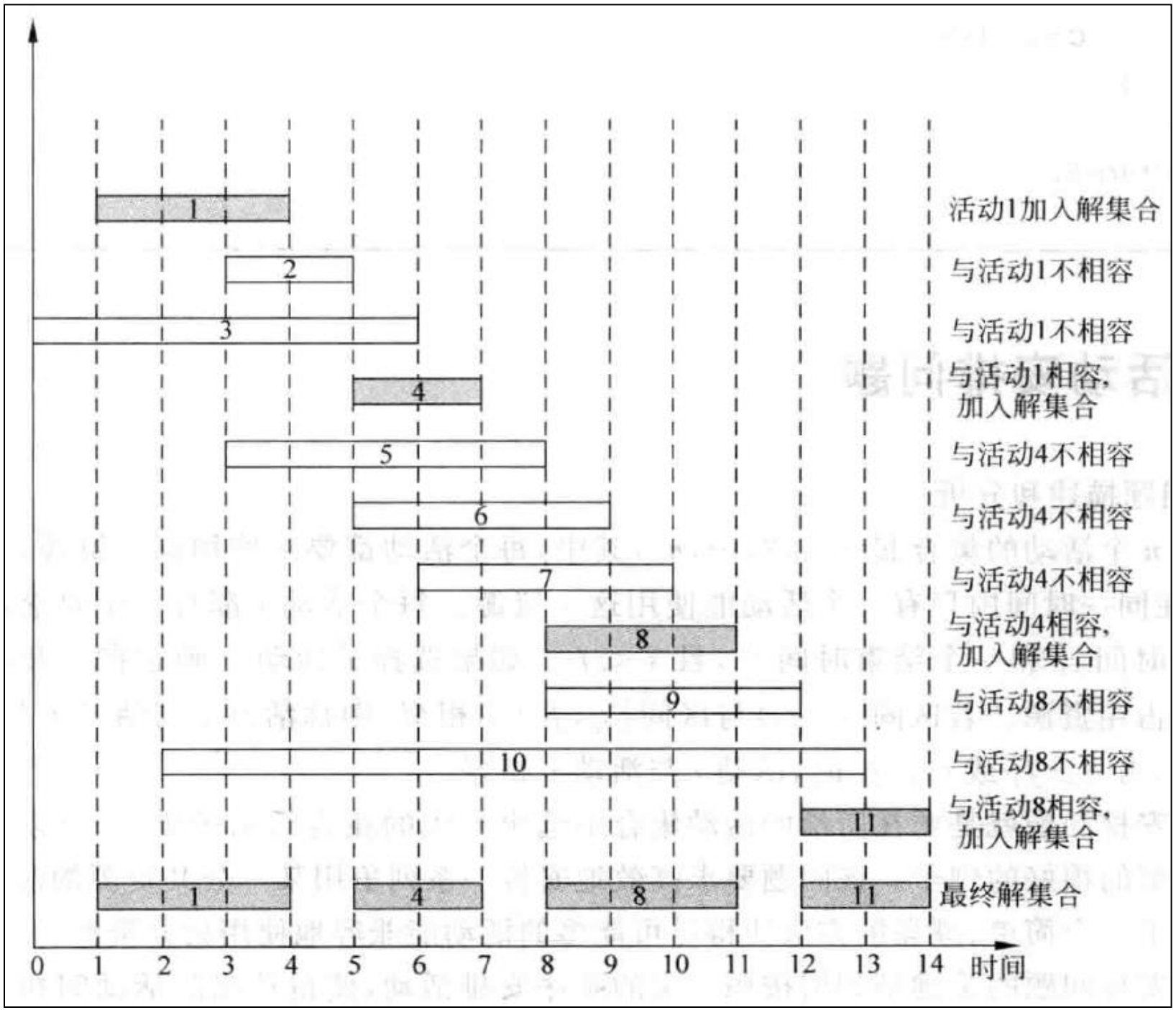
活动安排问题的例子

- 例子：求以下11个活动的最优安排（s_i:开始时间；f_i: 结束时间）

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

已按照结束时间非递减排序

活动安排问题例子的求解过程



活动安排问题的算法实现

```
int ActiveManage (int s[1..n ],int f[1..n ],bool a [1..n],int n)
{
    a[ 1] = 1;
    j = 1 ; count = 1 ;
    for (i = 2 ; i < = n; i ++)
    {
        if (s[ i] > = f[ j])
        {
            a [ i ] = 1 ;
            j = i;
            count ++;
        }
        else a [ i] = 0;
    }
    return count ;
}
```

已按照结束时间非递减排序

复杂度分析：

1、ActiveManage为 O(n)

2、对 f(n) 排序为 O(n*logn)

3、总体为 O(n*logn)

三、分数背包问题

0-1背包 vs. 分数背包 对比分析

贪心算法和动态规划算法都要求问题具有最优子结构性质，这是两种算法的共同点。但是，对于具有最优子结构的问题应该选用贪心算法还是动态规划算法求解?是否能用动态规划算法求解的问题也能用贪心算法求解?

回顾：**0-1背包问题（动态规划）**

- 给定n种物品和一个背包。物品i的重量是w_i，其价值为v_i，背包的容量为c。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？
- 在选择装入背包中的物品时，对每种物品i只有两种选择：即装入或不装入背包。不能将物品i装入背包多次，也不能只装入部分的物品i。因此，该问题称为0-1背包问题。

分数背包的贪心选择策略

0-1背包 vs. 分数背包　　对比：**分数背包（贪心算法）**

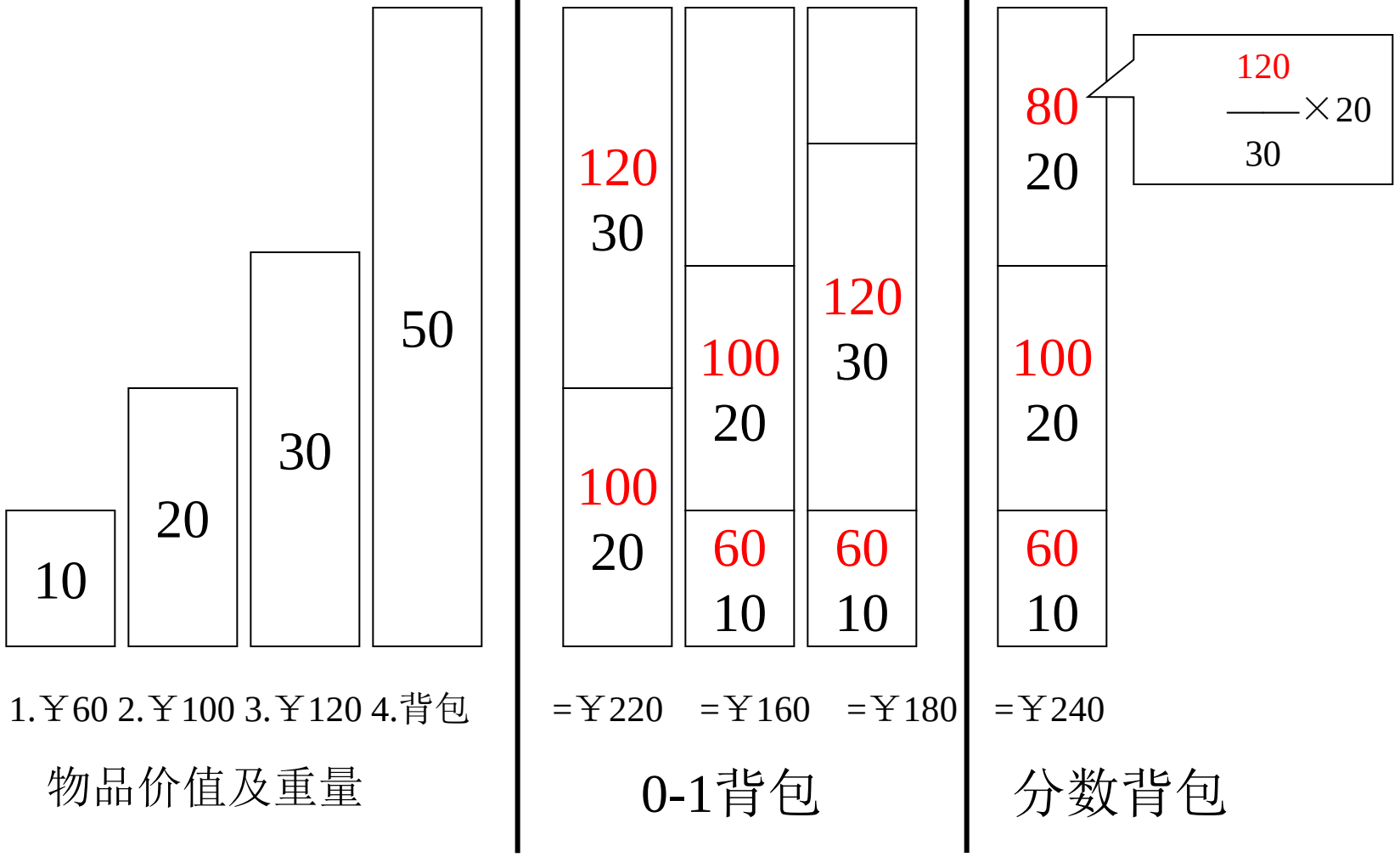
分数背包与0-1背包问题类似，所不同的是在选择物品 i 装入背包时，可以选择物品 i 的一部分，而不一定要全部装入背包。此问题的形式化描述为，给定 $c>0, w_i>0, v_i>0, 1\le i\le n$，要求找出一个 n 元0-1向量 $(x_1, x_2, \ldots, x_n)$，其中 $\mathbf{0\le x_i\le 1}$, $1\le i\le n$，使得对 w_ix_i 求和小于等于 c，并且对 v_ix_i 求和达到最大。

➢ **分数背包（贪心算法）**

- 首先计算每种物品单位重量的价值（$\mathbf{v_i/w_i}$）；
- 根据贪心选择策略将单位重量价值最高的物品装入背包。
- 若将这种物品全部装入背包后，背包内的物品总重量未超过 C，则选择单位重量价值次高的物品并尽可能多地装入背包。依此策略一直地进行下去，直到背包装满为止。

分数背包和0-1背包

0-1背包 vs. 分数背包



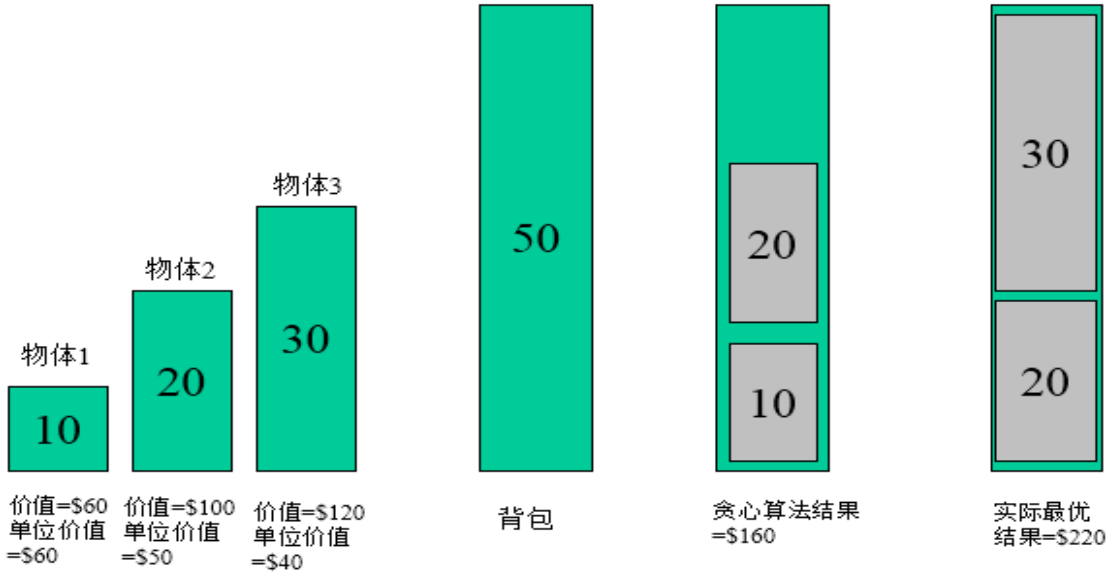
对比结论？

分数背包和0-1背包

0-1背包 vs. 分数背包

对于0-1背包问题，贪心选择之所以不能得到最优解是因为在这种情况下，它无法保证最终能将背包装满，部分闲置的背包空间使每公斤背包空间的价值降低了。而分数背包能用贪心算法得到最优解。

在考虑0-1背包问题时，应比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择（**从全局最优角度考虑，而不仅仅是当前最好**）。由此就导出许多互相重叠的子问题。这正是可用动态规划求解的另一重要特征，动态规划可以有效地解0-1背包问题.



分数背包的算法实现

```
float Knapsack(float w[0..n-1],float v[0..n-1],float x[0..n-1],float C,int n)
{
    for(i=0;i<n;i++)
        x[i]=0;
    i=1; total=0;
    while(w[i]<C)
    {
        x[i]=1;
        total=total+v[i];
        C=C-w[i];
        i++;
    }
    x[i]=C/w[i];
    total=total+x[i]*v[i];
    return total;
}
```

已按照 v_i/w_i 排序

X[i]可能是分数，确保背包一定可以装满

只取 X[i] 倍的 V[i] 的价值

复杂度分析：

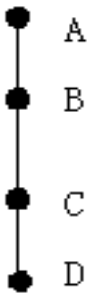
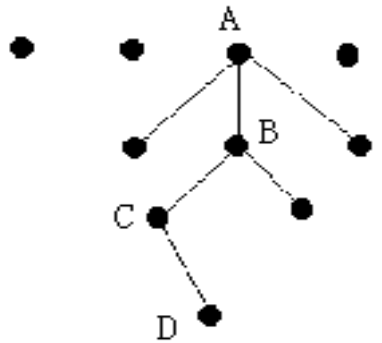
1、Knapsack 为 O(n)

2、排序为 O(n*logn)

3、总体为 O(n*logn)

贪心算法与动态规划差异

- 相同点：
  - 要求问题具备最优子结构性质。
- 不同点：
  - 1、动态规划算法的一个基本要素是重叠子问题性质，通常以自底向上的方式解各子问题；
  - 2、贪心算法通常以自顶向下的方式进行，以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

	贪心策略	动态规划
相同点	1 时间效率较高、适用于对问题的求解有严格时间限制的题目。 2 均能保证局部最优解。	
不同点	所占空间较小	所占空间较大
	某次具体选择必是最优选择	某次具体选择不一定是最优选择
	解题大体框架： 线形  A—B—C—D为最优选择，每一中间结点B、C也为最优选择	解题大体框架： 图或树  A—B—C—D为最优解，B、C不一定是最优选择

三、最小生成树问题

```
ESetType SpanningTree(ESetType E,int n)
{ //G=(V,E)为无向图，E是图G的边集，n是图中结点数
   ESetType TE=Ø;   //TE为生成树上边的集合
   int u,v,k=0; EType e;     //e=(u,v)为一条边
   while(k<n-1 && E中尚有未检查的边)
   {   //选择生成树的n-1条边
      e=select(E);  //按最优量度标准选择一条边
      if(TE∪e 不包含回路) //判定可行性
      {    //在生成树边集TE中添加一条边
         TE=TE∪e; k++;
      }
   }
   return TE;
}
```

- 将贪心策略用于求解无向连通图的最小代价生成树时，核心问题是需要确定贪心准则。
- 根据最优量度标准，算法的每一步从图中选择一条符合准则的边，共选择n-1条边，构成无向连通图的一棵生成树。
- 贪心法求解的关键：该量度标准必须足够好。它应当保证依据此准则选出n-1条边构成原图的一棵生成树，必定是最小代价生成树。

贪心选择标准，当前最优

3.1 最小生成树—Prim 算法

■ Prim算法的**贪心准则**：

在保证 TE 所代表的子图是一棵树的前提下选择一条最小代价的边e=(u,v)。

算法思想：

1. 在图G=(V, E) （V表示顶点集合，E表示边集合）中，从集合V中任取一个顶点（例如取顶点v_1）放入集合 U 中，这时 U={v_1}，生成树边集合TE 为空。
2. 寻找与 U 中顶点相邻（另一顶点在V-U中）权值最小的边的另一顶点 v_2，并使 v_2 加入U。即U={v_1,v_2 }，同时将该边加入集合 TE 中。
3. 重复2，直到U=V为止。

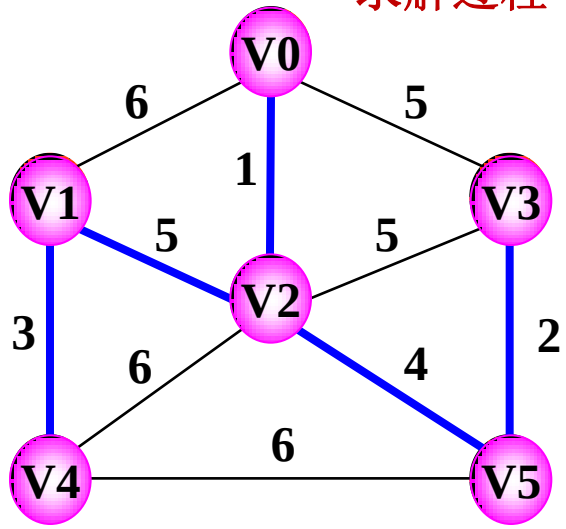
   这时 TE 中有 n-1 条边，T=(U, TE) 就是一棵最小生成树。

最小生成树-Prim算法

算法步骤：

- 设 $G=(V, E)$ 是连通图，TE 是 G 上最小生成树中边的集合。
- 初始令 $U=\{u_0\}$ $(u_0 \in V)$，$TE=\{\}$。
- 在所有 $u \in U, v \in V-U$ 的边 $(u, v) \in E$ 中，找一条代价最小的边 (u_0, v_0)。
- 将 (u_0, v_0) 并入集合 TE，同时 v_0 并入 U。
- 重复上述操作直至 $U=V$ 为止，则 $T=(V, TE)$ 为 G 的最小生成树。

求解过程



贪心选择标准，当前最优

最小生成树-Prim算法

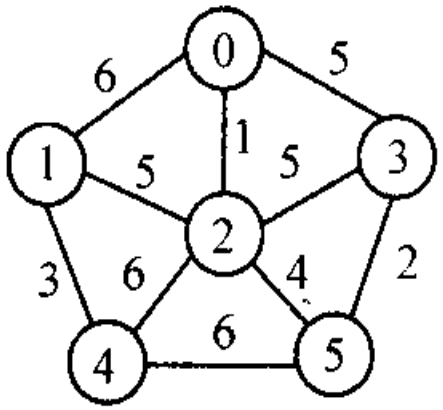
求解过程

V=?

U=?

V-U=?

TE=?



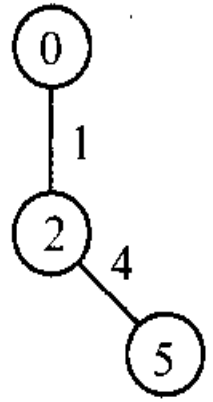
（a）无向图G



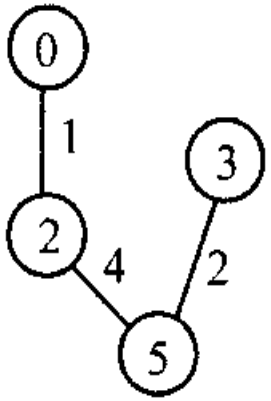
（b）只有源点



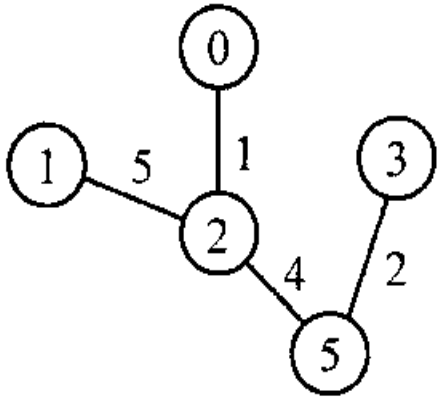
（c）加入第1条边



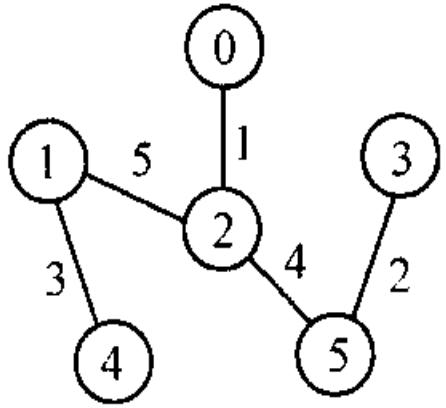
（d）加入第2条边



（e）加入第3条边



（f）加入第4条边



（g）图G的最小代价生成树

注意：Prim算法在选边的过程中，中间结果始终是一棵树

最小生成树-Prim算法

算法描述

设G=(V,E)是一个连通带权图, y={1, 2, …, n}。

算法思路: 首先置U={1}, T= Ø .若U⊂V, 就作如下的贪心选择：
选取满足条件 i∈U， j∈V-U，且 c[i][j] 最小的边(i, j)。将顶点 j 添加到 U 中，边 (i, j) 添加到 T 中。
这个过程一直进行到 U=V 时为止， T中的所有边构成G的一棵最小生成树。

```
void Prim(int n, Type  * * c)
 { T= Ø;
   U ={1};
    while (U!= V) {
          (i, j) = i ∈ U且 j∈V- U的最小权边;
          T=T ∪ {(i,j)};
           U= U ∪{j}; }
         }
```

关键步骤：如何选取满足条件 i∈U, j∈V-U,且 c[i][j] 最小的边 (i, j), 成了算法难点问题。

通过设置两个数组 closest 和 lowcost。对于每个 j ∈V-U，closet[j] 是 j 在 U 中的邻接顶点，它与 j 在 U 中的其他邻接顶点 k 相比较有 c[j][closet[j]] ≤c[j][k]。lowcost[j] 的值就是 c[j][closet[j]]。

最小生成树-Prim算法

Prim算法描述

```
template < class Type >
void Prim(int n, Type * * c)
{  Type lowcost[maxint];   //到U的最短距离
   int closest[ maxint];      //U中最短连接点
   bool s[maxint];
   s[1] = true;
   for(int i = 2;i<= n; i++){              //初始化
        lowcost[i] = c[l][i];
        closest[ i] = 1;
        s[i]= false;
   }
```

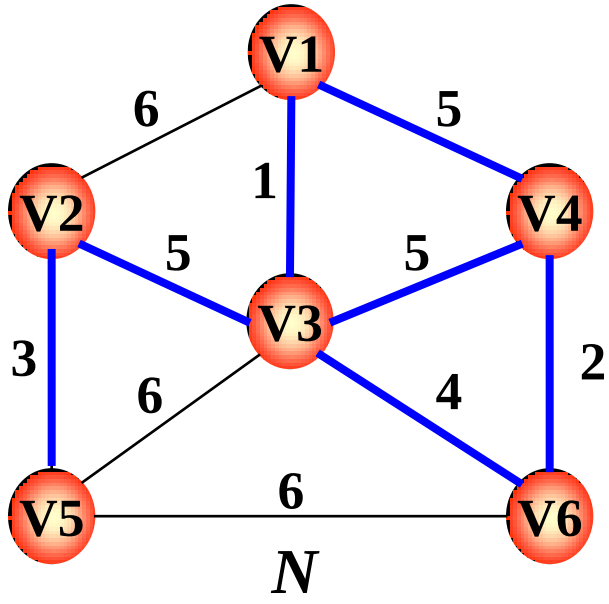
```
 for (int i = 1; i< n; i++)
{   Type min = infinite;
    int j = 1;
    for (int k = 2;k<= n; k++)
        if ((lowcost[k] < min)&&(!s[k]))
        {   min = lowcost[ k];
            j=k;    //搜索最近点
         };
    s[j] = true;
    for(int k= 2; k<= n; k++)
       if ((c[j][k] < lowcost[k])&&(!s[k]))
       { lowcost[k] = c[j][k]; //更新最短距离
         closest[ k] =j;
       }
  }
}
```

算法分析: $O(n^2)$

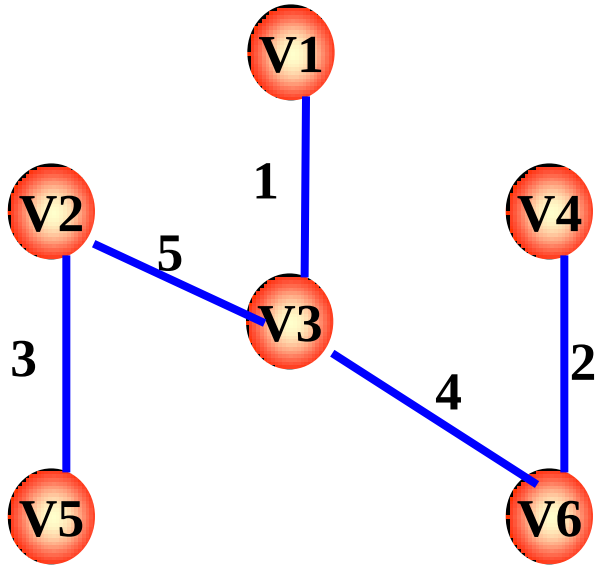
3.2 最小生成树-Kruskal算法

算法思想：

- 设连通网 $N=(V,E)$，令最小生成树初始状态为只有 n 个顶点而无边的非连通图 $T=(V,\{\})$，每个顶点自成一个连通分量。
- 在 E 中选取代价最小的边，若该边依附的顶点落在 T 中不同的连通分量上（即：不能形成环），则将此边加入到 T 中；否则，舍去此边，选取下一条代价最小的边。
- 依此类推，直至 T 中所有顶点都在同一连通分量上为止。



N



最小生成树-Kruskal算法

求解过程：

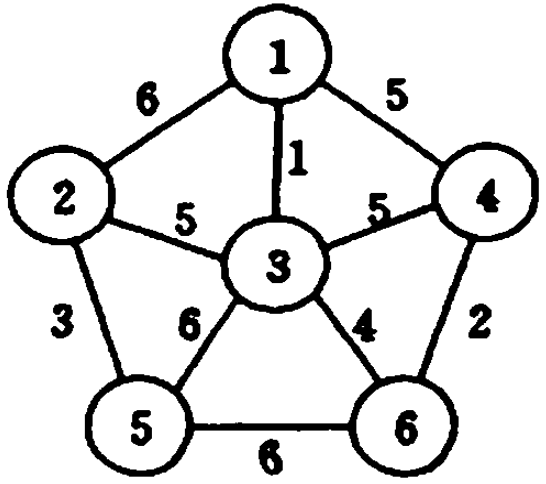


图 4 - 10　连通带权图

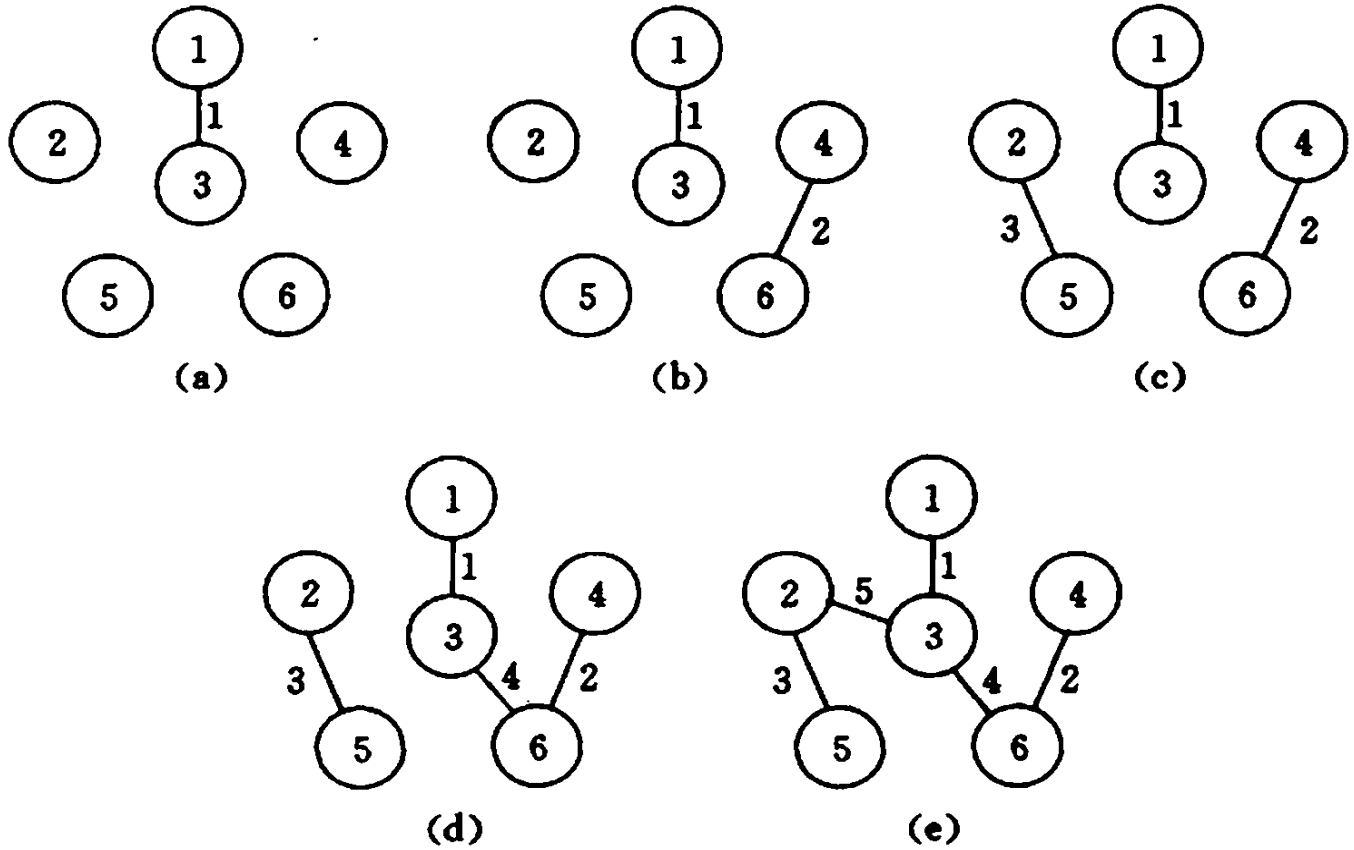


图 4 - 12　Kruskal 算法选边过程

最小生成树-Kruskal算法

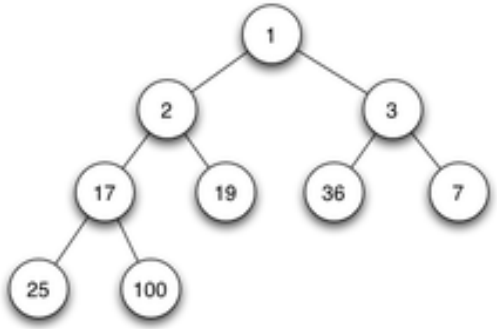
算法描述：

```
template < class Type >
bool Kruskal(int n, int e, EdgeNode < Type > E[ ], EdgeNode < Type > t[ ] )
        MinHeap < EdgeNode < Type > > H(1);
        H. Initialize(E, e);
        UnionFind U(n);  //独立分枝搜索对象
        int k = 0;
        while (e && k < n- 1) {
            EdgeNode <int > x;
            H. DeleteMin(x);//删除最小元素
            e--;
            int a = U.Find(x.u);
            int b = U.Eind(x.v);
            if (a! = b) {
               t[k ++] = x;//解集
          U. Union(a, b);}}
         H. Deactivate( );
        return (k = = n-1);
```

堆作为重要的**数据结构**之一，分为最大堆和最小堆是二叉堆 的两种形式。

1. 最大堆 ：根结点的键值是所有堆结点键值中最大者的堆。
2. 最小堆 ：根结点的键值是所有堆结点键值中最小者的堆。

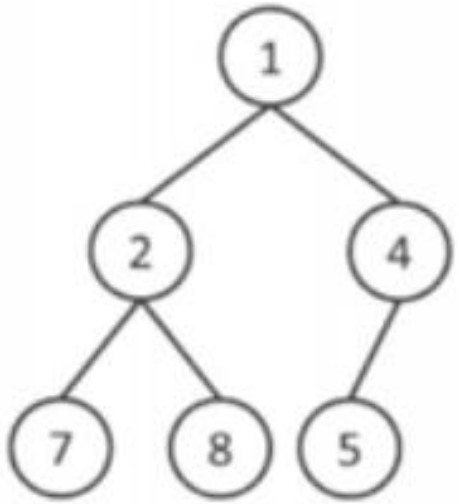
最小堆示例图：



算法分析:当图的边数为e时，将其组成优先队列需要时间O(e)while循环中,DeleteMin需要O(loge)时间因此关于优先队列所作运算需时间O(eloge)。实现UnionFind所需的时间为O(eloge)所以Kruskal的时间是O(eloge),适合求边数较少的最小生成树。

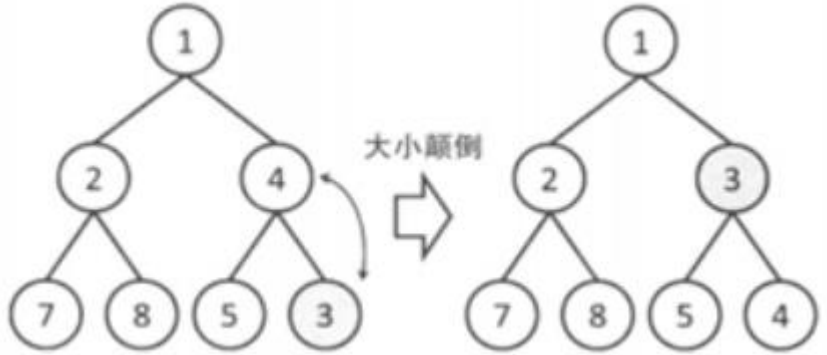
四、优先队列

- 能够完成以下两个操作的数据结构叫优先队列：
  - ◆可以插入新元素
  - ◆可以快速取出所有元素的最值（最大值或最小值）。
- 存储方式：
  - 一维线性表（一维数组），逻辑上组成一个二叉树。
- 对于某个节点x[i],有以下关系：
  - 根节点：x[1]
  - 父节点：x[i/2]
  - 左子节点：x[i*2]
  - 右子节点：x[i*2+1]

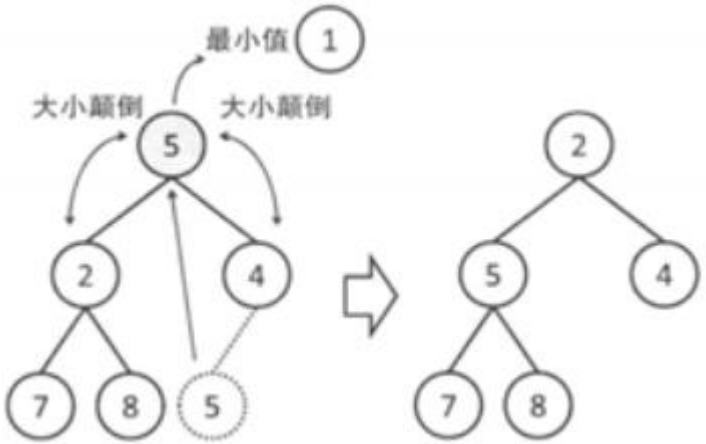


优先队列

- **堆的插入**：首先在堆的末尾插入该数值，然后不断向上调整，直到没有大小颠倒为止



- **取出最值**：最值就在堆顶，即二叉树的第一个元素。
- **删除最值**：首先将堆的最后一个元素复杂到根节点，并删除最后一个元素，然后将根节点不断向下进行调整直到没有大小颠倒



五、哈夫曼编码

1951年，哈夫曼和他在MIT信息论的同学需要选择是完成学期报告还是期末考试。导师Robert M. Fano给他们的学期报告的题目是，寻找最有效的二进制编码。由于无法证明哪个已有编码是最有效的，哈夫曼放弃对已有编码的研究，转向新的探索，最终发现了基于有序频率二叉树编码的想法，并很快证明了这个方法是最有效的。

- 哈夫曼编码是广泛地用于数据文件压缩的十分有效的编码方法。其压缩率通常在20%～90%之间。
- 哈夫曼编码算法是用字符在文件中出现的频率表来建立一个用0,1串表示各字符的最优表示方式。
- 编码目标：给出现频率高的字符较短的编码，出现频率较低的字符以较长的编码，可以大大缩短总码长。

哈夫曼编码-引例

一个数据文件包含100000个字符，要用压缩的方式来存储它。该文件中各字符出现的频率如表所示。

	a	b	c	d	e	f
频率(千次)	45	13	12	16	9	5
定长码	000	001	010	011	100	101
变长码	0	101	100	111	1101	1100

使用变长码要比使用定长码好得多。通过给出现频率高的字符较短的编码，出现频率较低的字符以较长的编码，可以大大缩短总码长。

哈夫曼编码-前缀码

- 定义：对每一个字符规定一个0,1串作为其代码，并要求任一字符的代码都不是其他字符代码的前缀。这种编码称为前缀码。
- 编码的前缀性质可以使译码方法非常简单。由于任一字符的代码都不是其他字符代码的前缀，从编码文件中不断取出代表某一字符的前缀码，转换为原字符，即可逐个译出文件中的所有字符。

给定序列译码？001011101

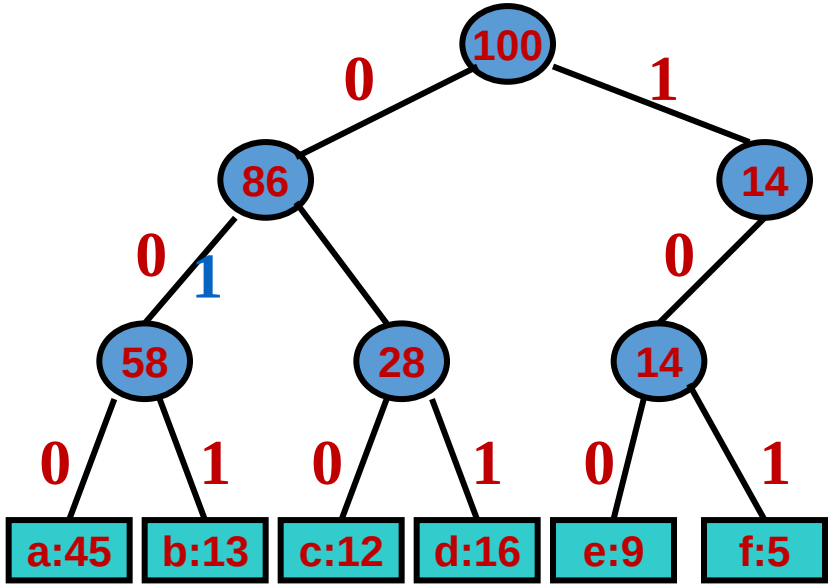
可唯一地分解为0,0,101,1101，译码为aabe

	a	b	c	d	e	f
频率(千次)	45	13	12	16	9	5
定长码	000	001	010	011	100	101
变长码	0	101	100	111	1101	1100

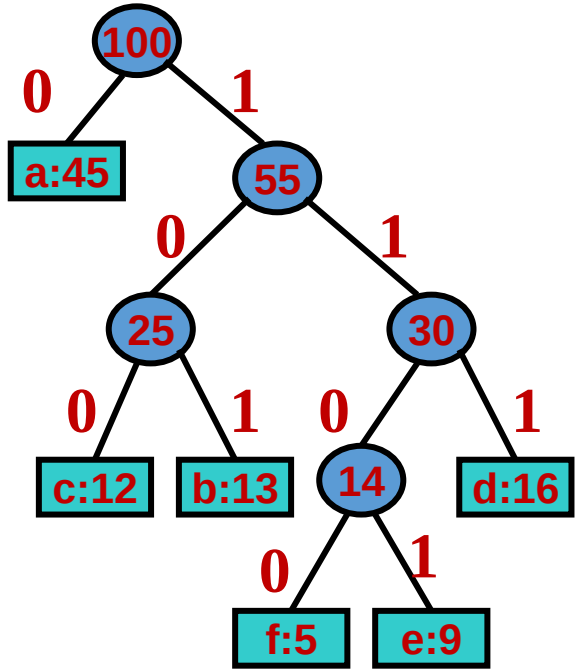
哈夫曼编码-问题分析

- 译码过程需要方便地取出编码的前缀，因此需要一个表示前缀码的合适的数据结构。
- 用二叉树作为前缀编码的数据结构。在表示前缀码的二叉树中，树叶代表给定的字符，并将每个字符的前缀码看作是从树根到代表该字符的树叶的一条道路。代码中每一位的0或1分别作为指示某结点到其左儿子或右儿子的“路标”。

定长码



变长码



哈夫曼编码-算法思路

- 哈夫曼算法以自底向上的方式构造表示最优前缀码的二叉树T。
- 编码字符集中每一字符c的频率是 $f(c)$。以f为键值的优先队列Q用以在作贪心选择时有效地确定算法当前要合并的两棵具有最小频率的树。一旦两棵具有最小频率的树合并后，产生一棵新的树，其频率为合并的两棵树的频率之和，并将新树插入优先队列Q中，再进行新的合并。

1) 以n个字母为结点构成n棵仅含一个点的二叉树集合,字母的频率即为结点的权
2) 每次从二叉树集合中找出两个权最小者合并为一棵二叉树: 增加一个根结点将这两棵树作为左右子树.新树的权为两棵子树的权之和.
3) 反复进行步骤2)直到只剩一棵树为止.

编码方案的平均码长定义为：

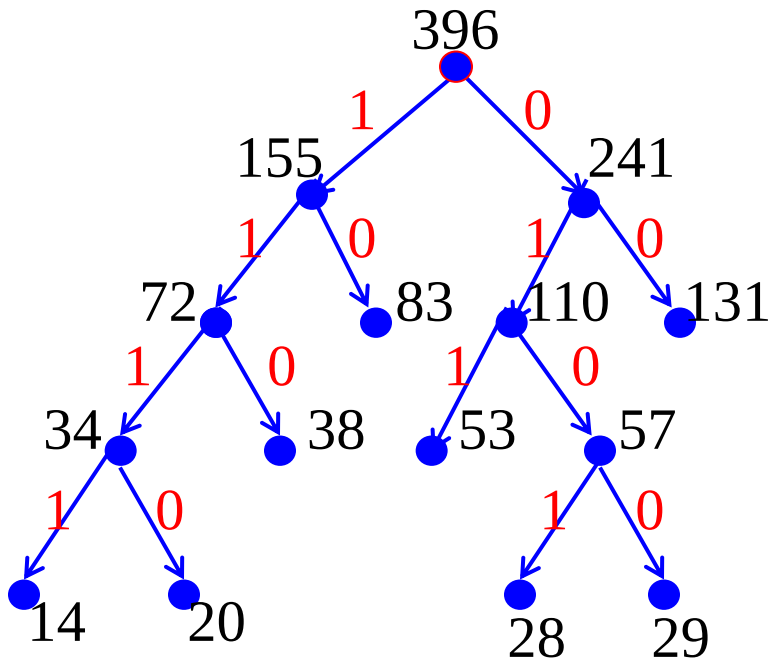
$$B(T)=\sum_{c\in C} f(c)d_T(c)$$

哈夫曼编码-算法实例

设在1000个字母的文章中各字母出现的频率为: a:83, b:14, c:28, d:38, e:131, f:29, g:20, h:53.....

14	20	28	29	38	53	83	131
	34	28	29	38	53	83	131
		34	57	38	53	83	131
			57	72	53	83	131
				72	110	83	131
					110	155	131
						155	241
							396

1) 将权从小到大排序
2) 每次选取最小权合并



最佳编码:

a:10 ; b:1111; c:0101; d:110;

e:00; f:0100; g:1110; h:011

哈夫曼编码-算法描述

```
template<class T>
BinaryTree<int> HuffmanTree(T f[], int n)
{ //根据权f[1:n]构造哈夫曼树
  //创建一个单节点树的数组
 Huffman <T>* W=new Huffman<T> [n+1];
  BinaryTree<int>  z, zero;
  for(int i=1;  i<=n;  i++) {
      z.MakeTree(i, zero, zero);  //zero空树
      W[i].weight=f[i];  //初始化霍夫曼树
      W[i].tree=z: }
 //数组变成一个最小堆
  MinHeap<Huffman<T>> Q(1);
  Q.Initialize(W,n,n);  //初始化Heap
  //将堆中的树不断合并
  Huffman<T> x, y;
for(i=1;i<n;i++){
     Q.DeleteMin(x);
      Q.DeleteMin(y);
      z.MakeTree(0, x.tree, y.tree);
      x.weight+=y.weight;  x.tree=z;
      Q.Insert(x);   }
 Q. DeleteMin(x);  //最后的树
 Q. Deactivate();
 delete[] w;
 return x.tree;
```

六、最短路问题

最短路问题是指给定一个边带权的图G，求从某个起始点到某个终点e的权值和最小的路径。

应用：在一个交通运输网络中求城市之间的最短运输路径，求互联网中信号传输的最小延迟等。

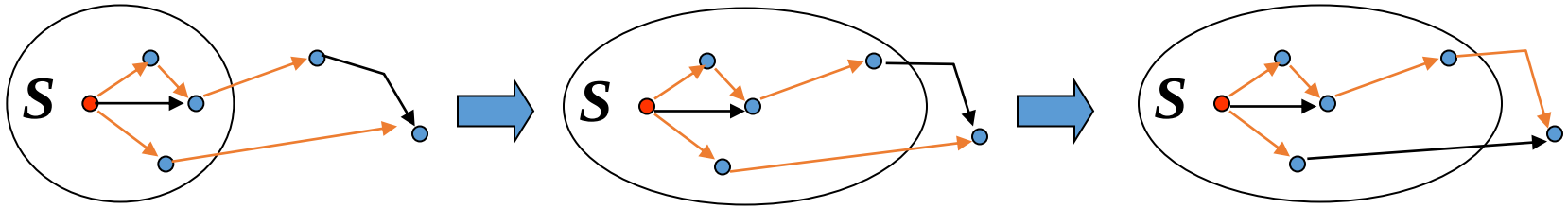
最短路的Dijkstra算法

问题描述: 给定带权有向图G=(V,E), 其中每条边的权是一个非负实数.要计算从V的一点v_0(源)到所有其他各顶点的最短路长度. 路长指路上各边权之和。



算法思路(Dijkstra) :设最短路长已知的终点集合为S,初始时$v_0 \in S$, 其最短路长为0,然后用贪心选择逐步扩充S :每次在V-S中,选择**路径长度值最小的一条最短路径**的终点x加入S.

构造路长最短的最短路径:设已构造 i 条最短路径,则下一个加入S的终点u必是V-S中具有最小路径长度的终点, 其长度或者是弧(v_0,u), 或者是中间只经过S中的顶点而最后到达顶点u的路径.



最短路Dijkstra算法

- Dijkstra算法是求解单源最短路径问题的一个贪心算法。
- 基本思想：设置一个顶点集合S ，并不断地作贪心选择来扩充这个集合。一个顶点属于集合 S 当且仅当从源到该顶点的最短路径长度已知。
- Dijkstra算法通过分步方法求出最短路径。
  - 每一步产生一个到达新的目的顶点的最短路径。
  - 下一步所能达到的目的顶点通过这样的贪心准则选取：在还未产生最短路径的顶点中，选择路径长度最短的目的顶点。
- 也就是说， Dijkstra算法按路径长度顺序产生最短路径。
- 实际上，下一条最短路径总是由已产生的最短路径再扩充一条最短的边得到的，且这条路径所到达的顶点其最短路径还未产生。

最短路Dijkstra算法

算法步骤

(1) 用带权的邻接矩阵c来表示带权有向图, c[i][j]表示弧<vi,vj>上的权值. 若 <vi, vj>∉V,则置c[i][j]为∞.

设S为已知最短路径的终点的集合,它的初始状态为空集.

从源点v*经过S* 到图上其余各点vi的当前最短路径长度的初值为：

dist[i]=c[v][i], vi∈V.

(2) 选择vj, 使得dist[j]=Min{dist[i] | vi∈V-S},vj就是长度最短的最短路径的终点。令S=S U {j}.

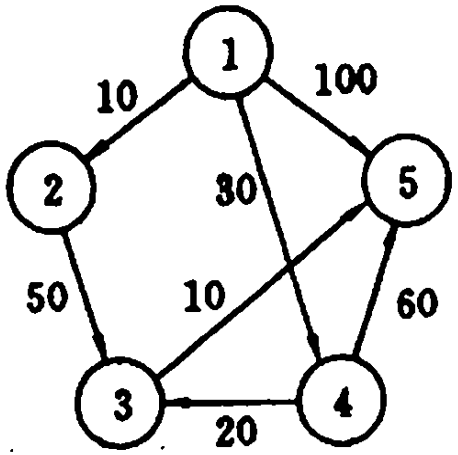
(3) 修改从v到集合V-S上任一顶点v_k的当前最短路径长度:

如果 dist[j]+c[j][k]< dist[k] 则修改 dist[k]= dist[j]+c[j][k].

(4) 重复操作(2),(3)共n-1次.

最短路Dijkstra算法

计算过程分析

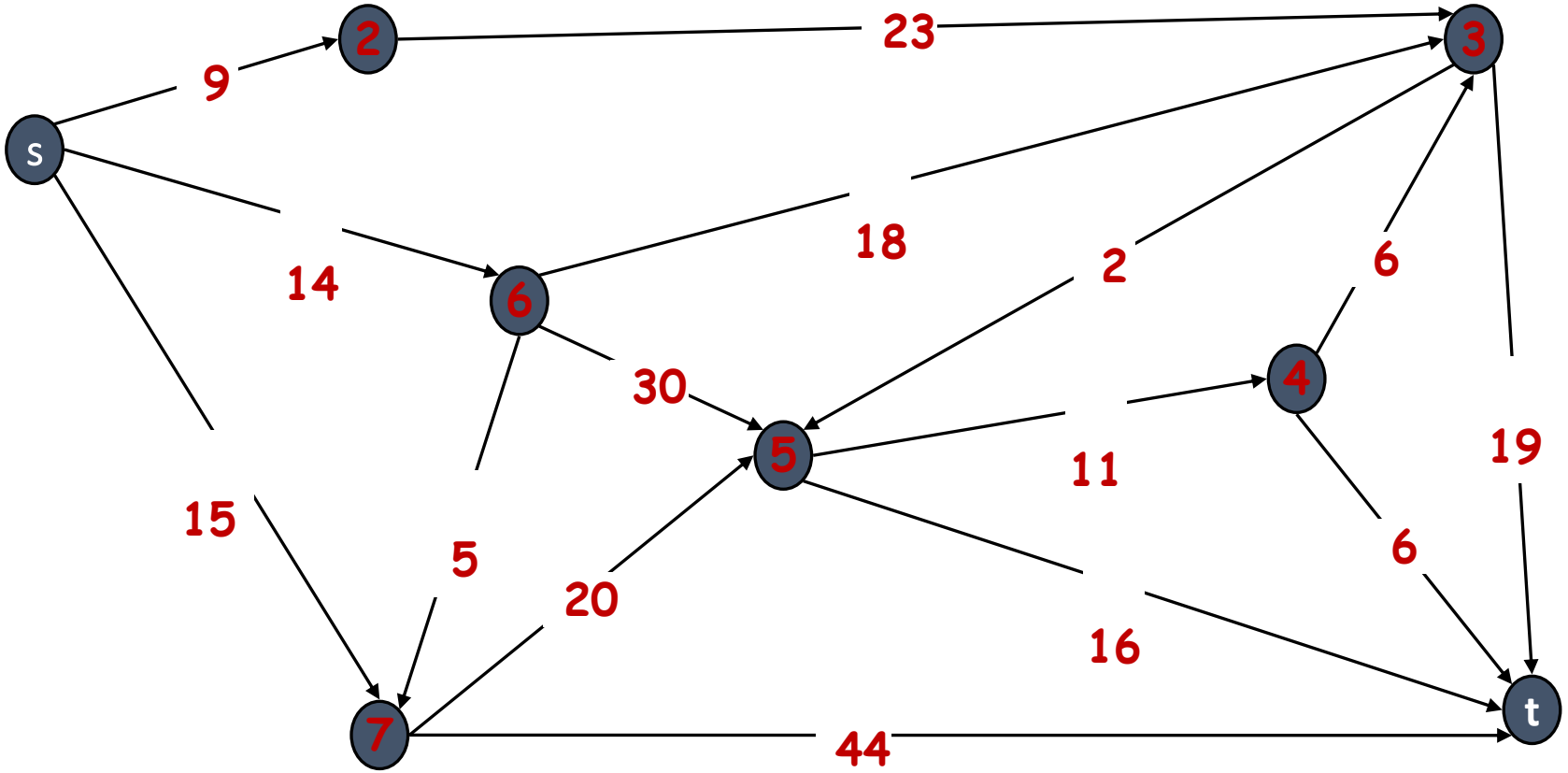


∞	10	∞	30	100
∞	∞	50	∞	∞
∞	∞	∞	∞	10
∞	∞	20	∞	60
∞	∞	∞	∞	∞

迭代	S	vi	d[2]	d[3]	d[4]	d[5]
初始	{1}		10	∝	30	100
1	{1,2}	2	*10*	60	30	100
2	{1,2,4}	4	*10*	50	*30*	90
3	{1,2,3,4}	3	*10*	*50*	*30*	60
4	{1,2,3,4,5}	5	*10*	*50*	*30*	*60*

最短路Dijkstra算法

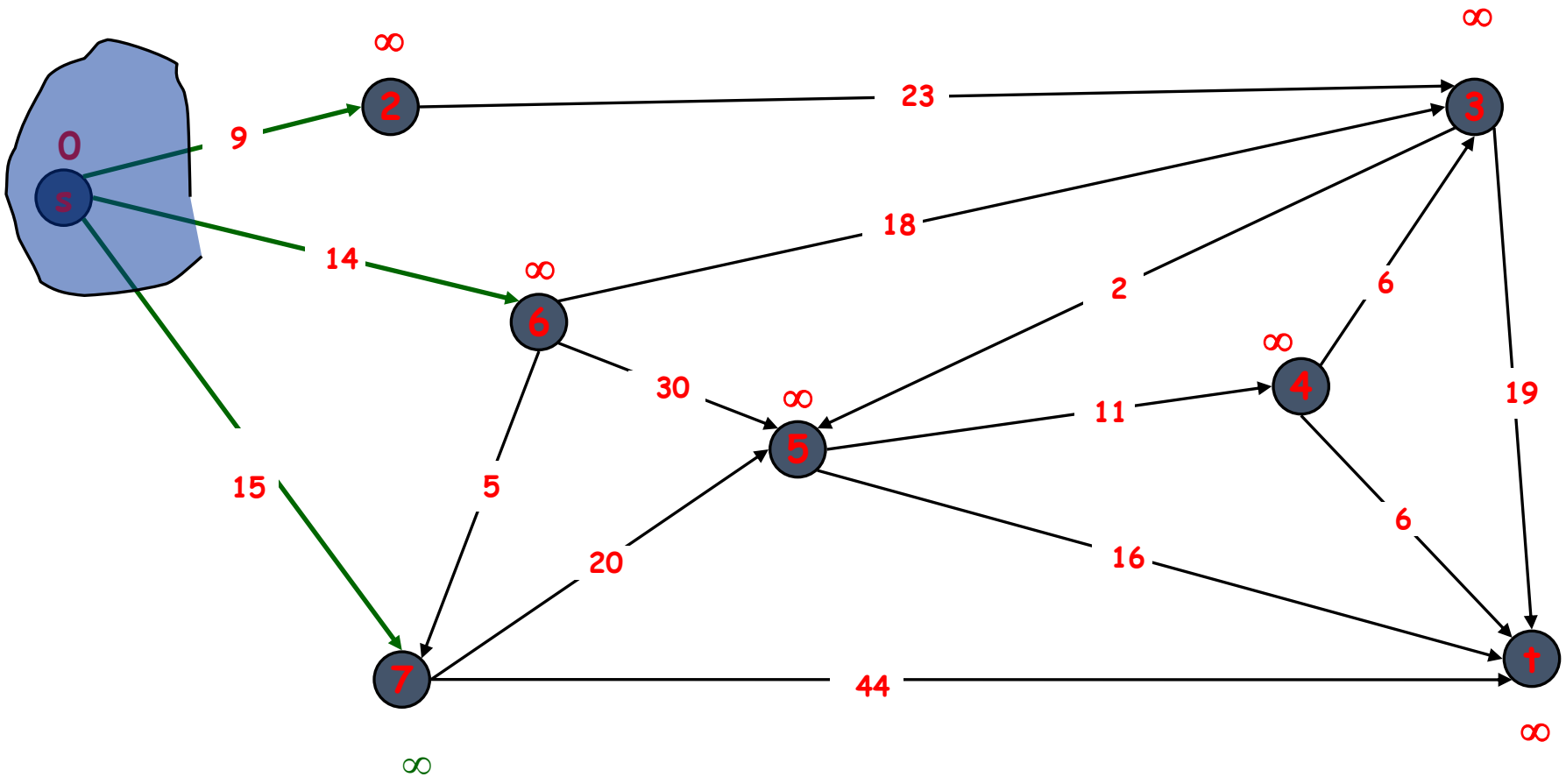
计算过程实例



最短路Dijkstra算法

计算过程实例

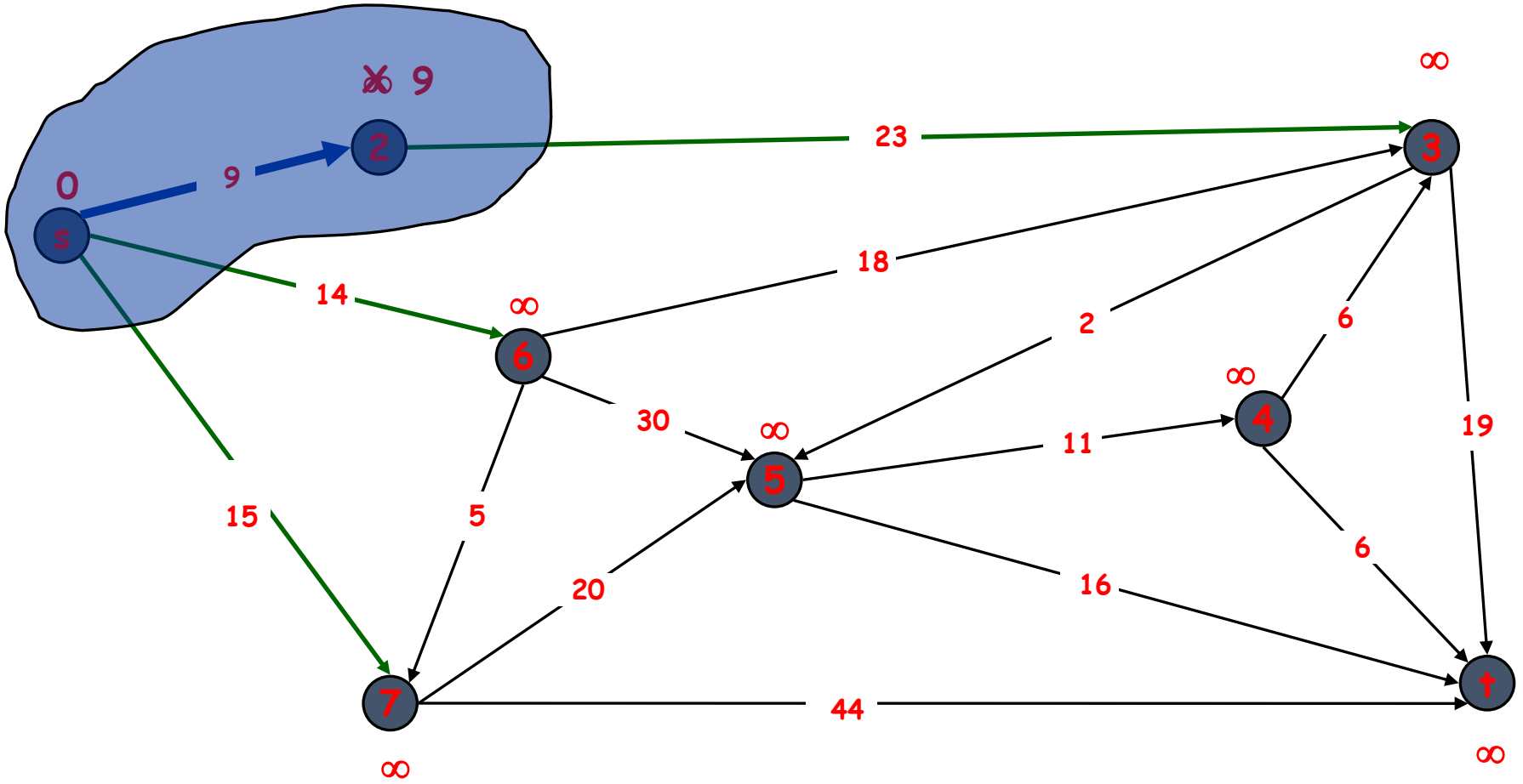
S={ s }



最短路Dijkstra算法

计算过程实例

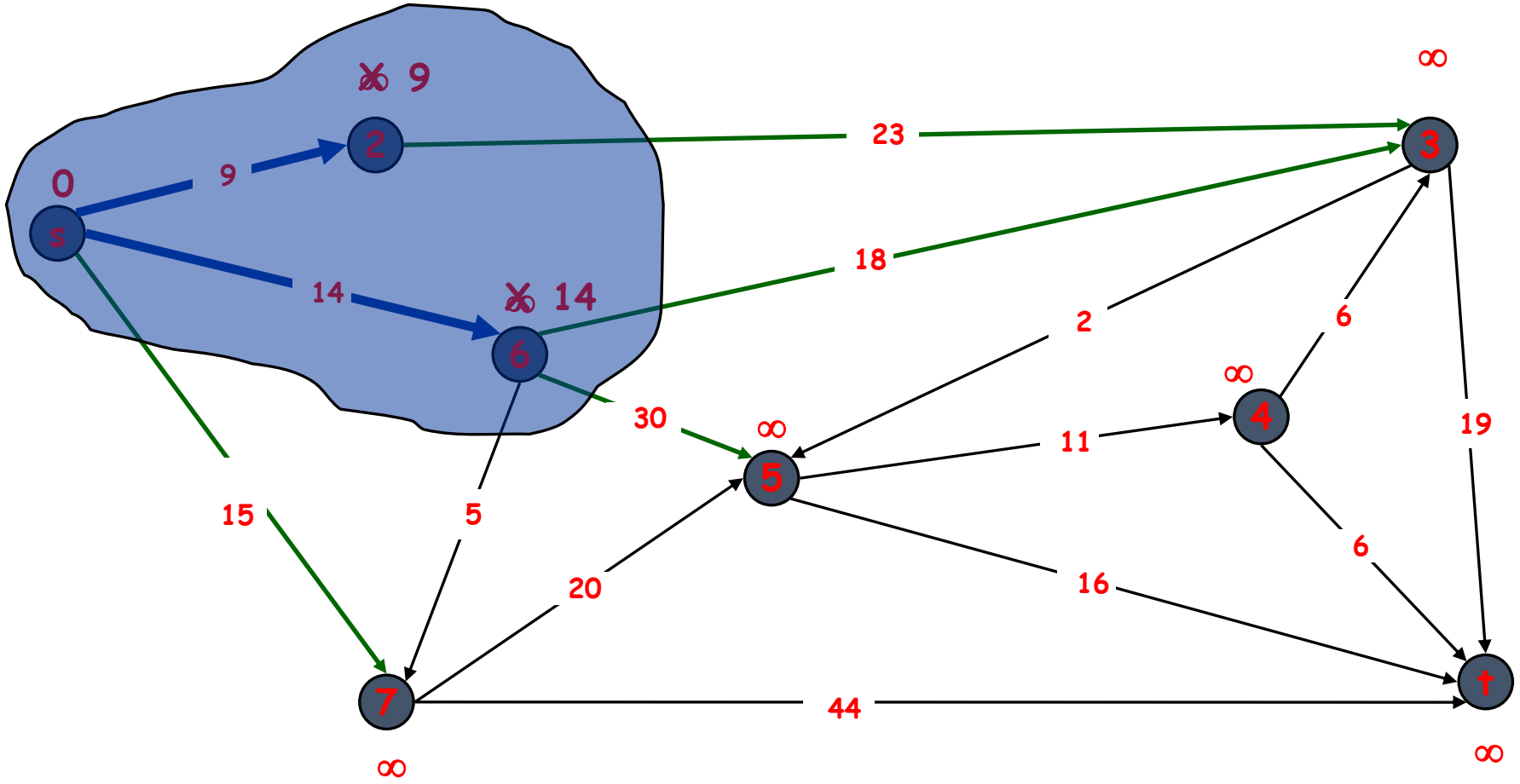
S = { s, 2 }



最短路Dijkstra算法

计算过程实例

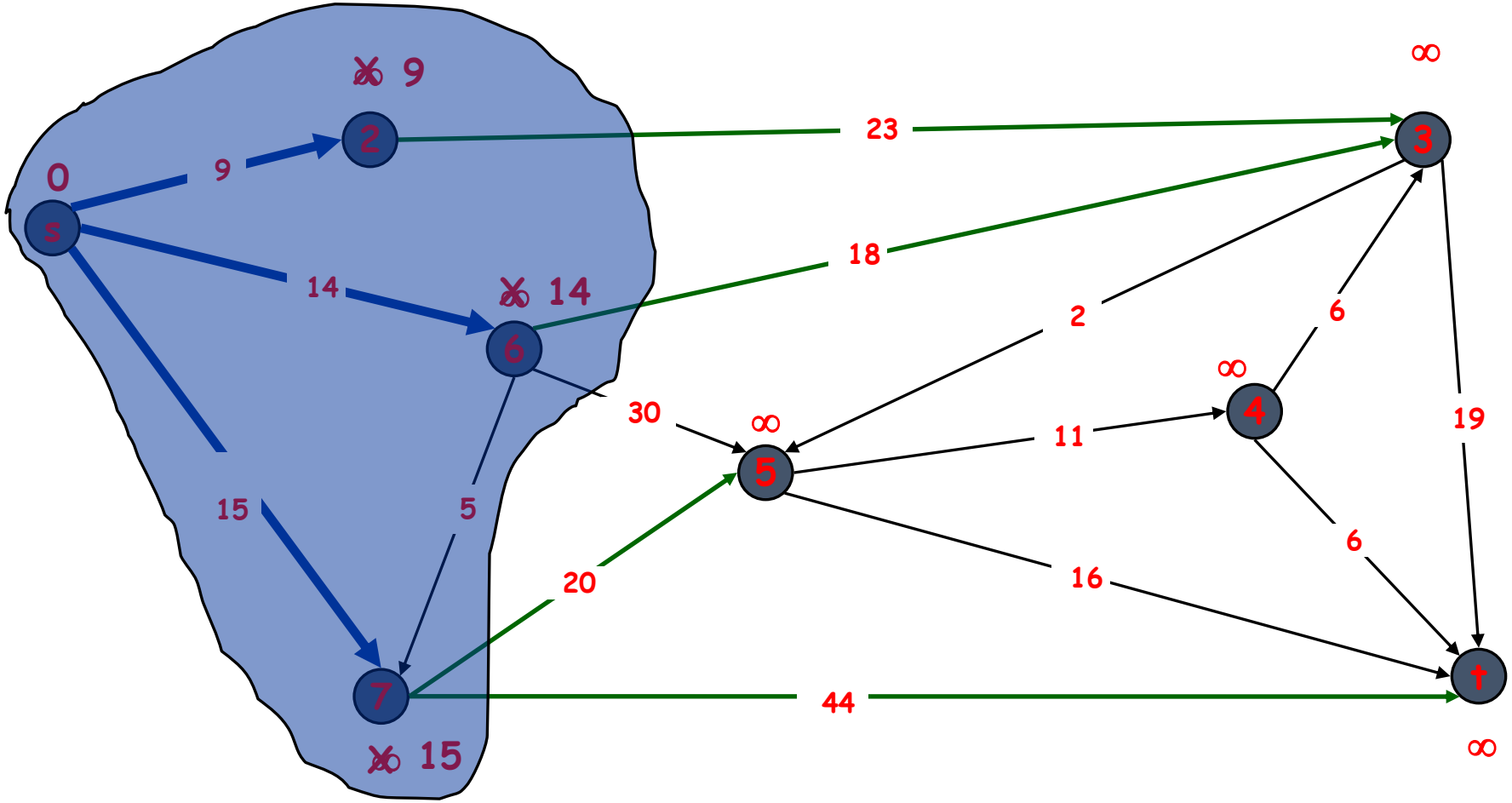
S = { s, 2, 6 }



最短路Dijkstra算法

计算过程实例

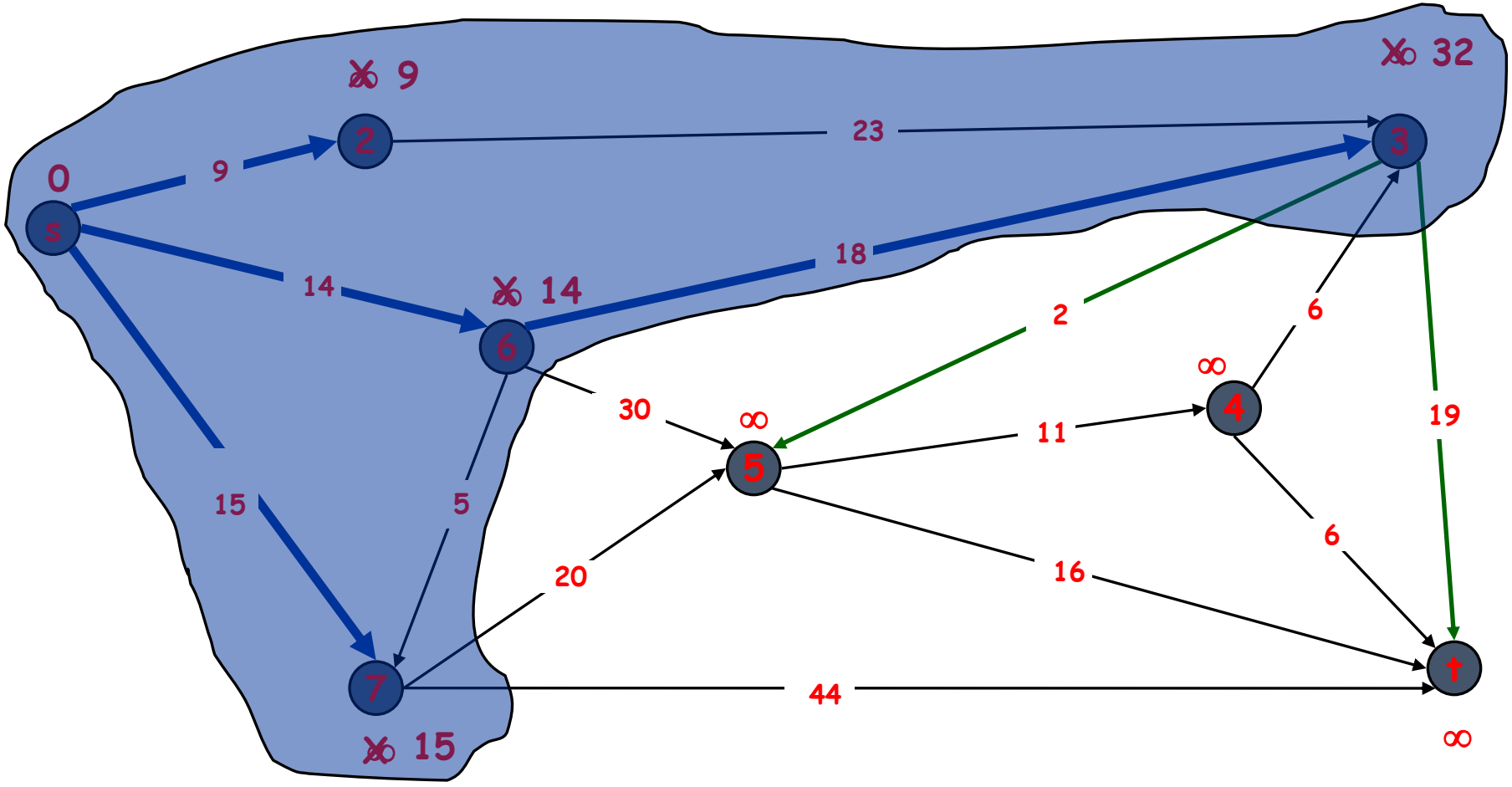
S = { s, 2, 6, 7 }



最短路Dijkstra算法

计算过程实例

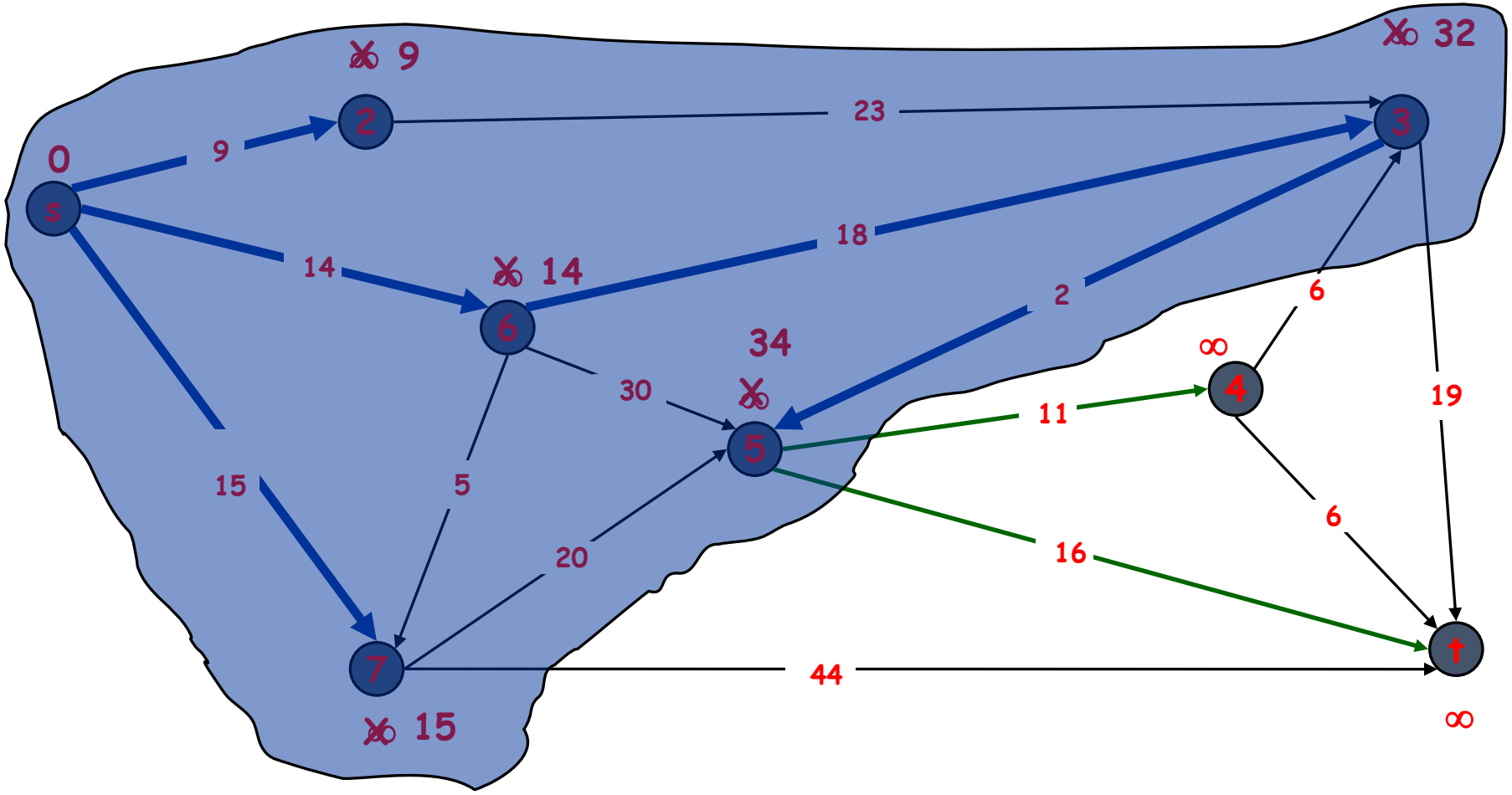
S = { s, 2, 3, 6, 7 }



最短路Dijkstra算法

计算过程实例

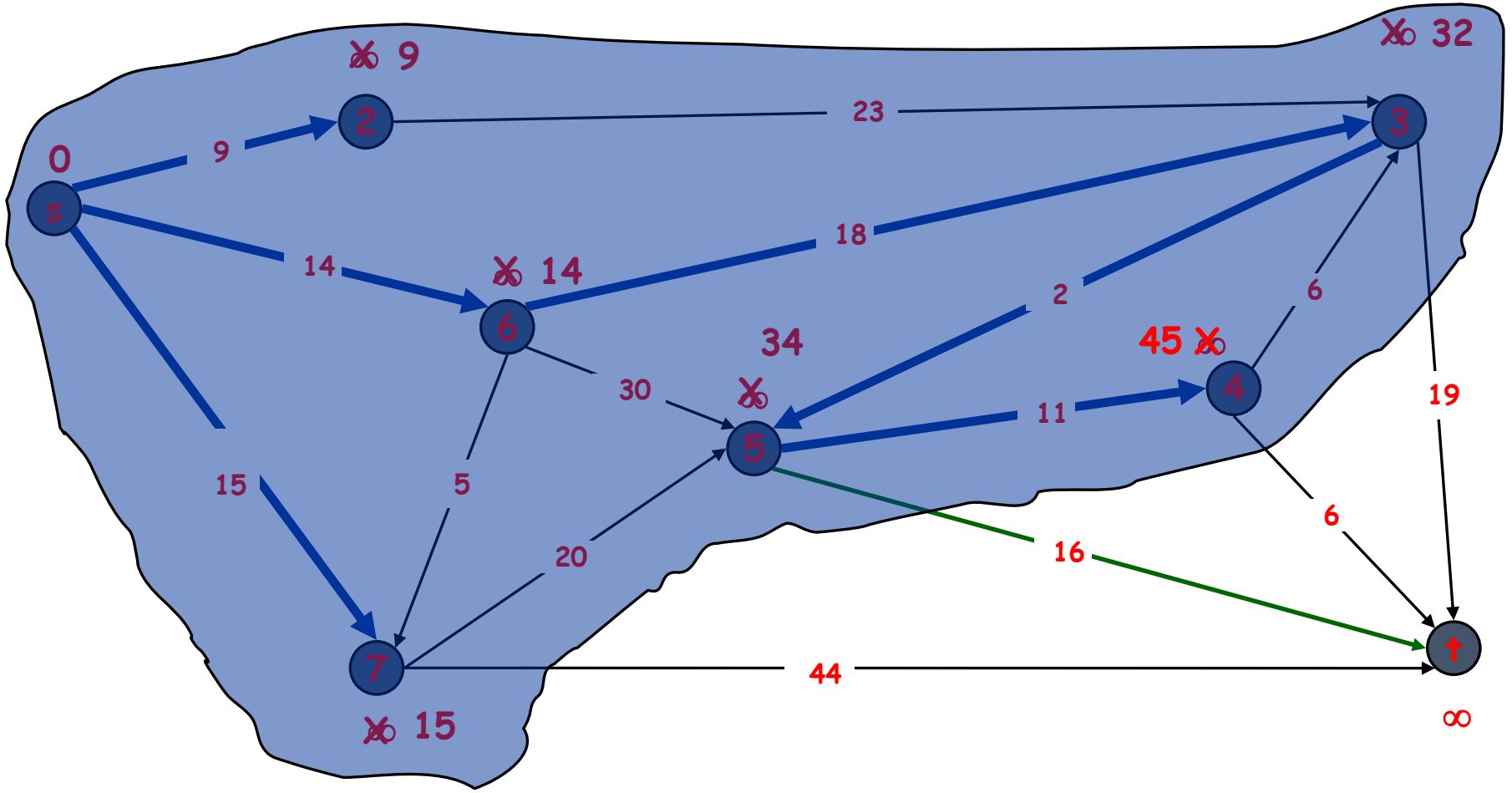
S = { s, 2, 3, 5, 6, 7 }



最短路Dijkstra算法

计算过程实例

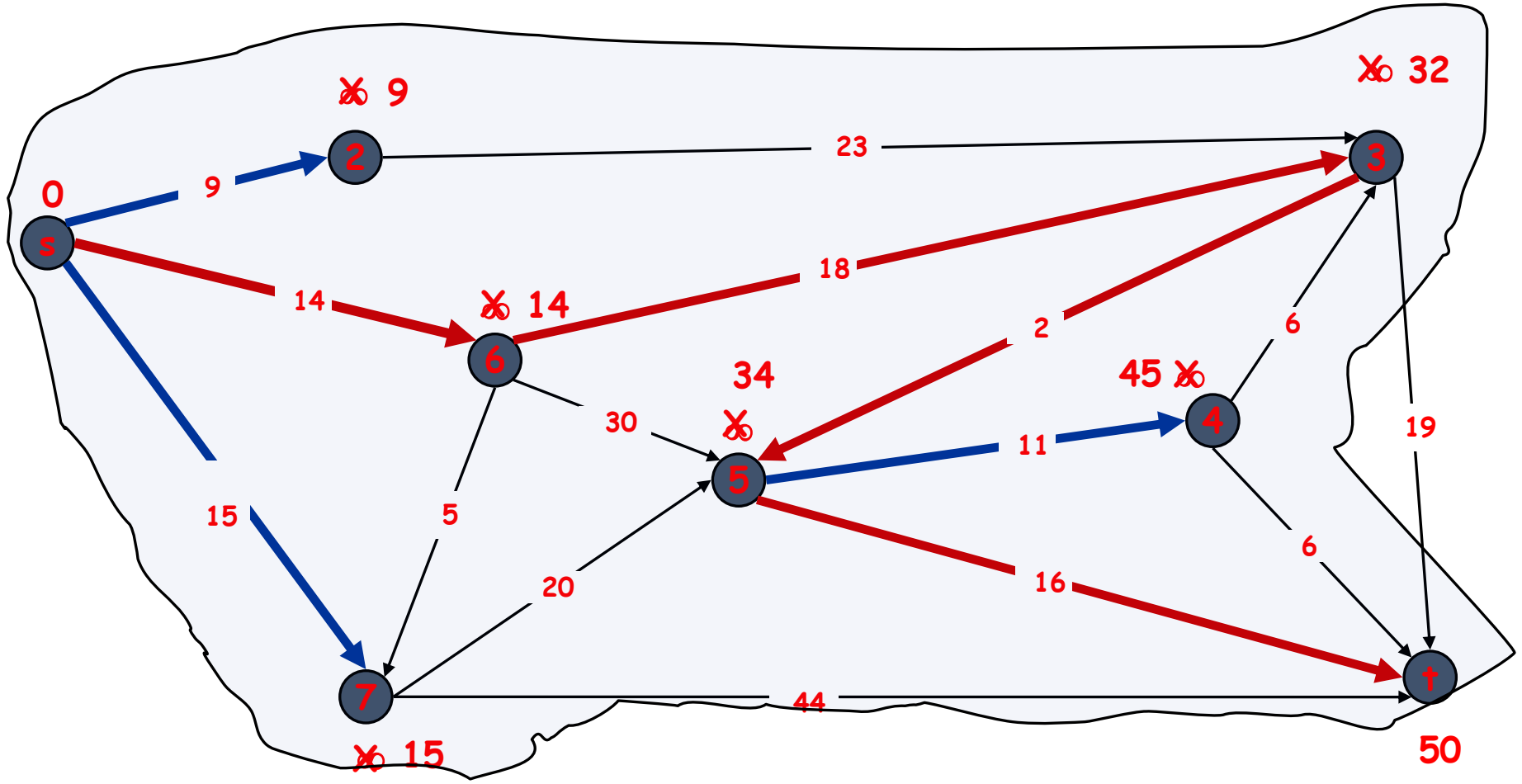
S = { s, 2, 3, 4, 5, 6, 7 }
Q = { t }



最短路Dijkstra算法

计算过程实例

S = { s, 2, 3, 4, 5, 6, 7, t }
Q = { }



最短路Dijkstra算法

算法描述

```
void Dijkstra(int n, int v,Type dist[], int prev[], Type **c)
{ bool s[maxint];
  for (int i=1;i<=n; i++) {
    dist[i]=c[v][i]; //边权
    s[i]=false;
    if(dist[i]= =maxint) prev[i]=0;
    else   prev[i]=v ; //i的前一个顶点}
  dist[v]=0;  s[v]=true; //初始化v顶点属于S集
  for (int i=1;  i<n;  i++){
    int temp=maxint;
    int u=v;
    for (int j = 1;  j<=n;  j++)
      if ((!s[j])&&(dist[j]<temp)){
        u=j;       //搜索S外离v最近点
        temp=dist[j];  }
    s[u]=true;       //将u放入S中
    for (int j=1;  j<=n;j++)
      if((!s[j])&&(c[u][j]<maxint)){
        Type  newdist=dist[u]+c[u][j];
      if (newdist<dist[j]){
        dist[j]=newdist;
        prev[j]=u;  }}}}
```

分析:用带权邻接矩阵表示有n个顶点和e条边的带权有向图,主循环体需要O(n)时间,循环需要执行n-1次,所以完成循环需要$O(n^2)$.

2. 任意边权的单源最短路

当边权可能为负值的时候，使用Dijkstra算法不一定能得到正确的结果。所以可以采用Bellman-Ford(贝尔曼-福特）算法。该算法**要求图中不能存在边权和为负数的回路**。

算法描述：

- 计算一个边数递增的最短路序列$D_1[u]$，$D_2[u]$，… $D_{n-1}[u]$ 。
- $D_i[u]$表示从起始点s出发到达顶点u最多经过i条边的最短路的长度。
- 这样，最终的计算结果$D_{n-1}[u]$ 就是问题的答案。

最短路其他算法

递推方法：

假设已经求出了$D_{k-1}[u]$。 即已得到了从起始点s到达顶点j最多经过k-1条边的最短路，而从顶点j又存在一条边可以到达顶点u，那么从起始点s到达顶点u最多经过k条边的最短路可能是先从s出发经过最多k-1条边到达j，然后再由j经过一条边到达u。只要枚举所有的j，取其中最小值，即可计算得出$D_k[u]$。

递推公式：

$D_1[u]$=起始点s到顶点u的直接边的权值

$D_k[u]=\min\{D_{k-1}[u],\min\{D_{k-1}[j]+$顶点j到顶点u的直接边的权值$\}\}$

$(0<=j<n)$

已学算法策略小结

- 分治法
  - 将复杂问题分成若干个相互独立的子问题，通过求解子问题，并将子问题的解合并得到原问题的解。
- 动态规划法
  - 将一个复杂问题分解为若干个相互重叠的子问题，通过求解子问题形成一系列决策得到原问题的解。
  - 动态规划是对分治的改善，在发现有重叠问题时，使用自底向上的策略避免重复计算，从而提升了算法的效率。
- 贪心法
  - 将一个复杂问题分解为一系列较为简单的局部最优选择，每一个选择都是对当前解的一个扩展，直到获得问题的完整解。



课堂练习：贪心算法求解汽车加油问题

问题描述

一辆汽车加满油后可行驶 n *km* 。旅途中有若干个加油站。设计一个有效算法，指出应在哪些加油站停靠加油，使沿途加油次数最少。

编程任务

对于给定的n和k个加油站位置，编程计算最少加油次数。

课堂练习：贪心算法求解汽车加油问题

数据输入

第1行有2个正整数n和k，表示汽车加满油后可行驶n km，且旅途有k个加油站。接下来的一行中，有k+1个整数，表示第k个加油站与第k-1个加油站之间的距离。第0个加油站表示出发地，汽车已加满油。第k+1个加油站表示目的地。

结果输出

计算出的最少加油次数。如果无法到达目的地，则输出“No Solution”。

输入示例

```
7 7
1 2 3 4 5 1 6 6
```

输出示例

```
4
```

k	0	1	2	3	4	5	6	7
x	1	2	3	4	5	1	6	6

课堂练习：贪心算法求解汽车加油问题

```
int greedy(vector<int> x,int n)
{
    int j,i,s,sum=0, k=x.size();
    for(j=0;j<k;++j)        //k-加油站个数
        if(x[j] > n)    {   //x-加油站之间距离
            cout<<"No Soultion"<<endl;
            return -1;
        }
    for(i=0,s=0;i <k; ++i)    {
        s += x[i];
        if(s > n) sum++,s = x[i];
    }
    return sum;
}
```

i=3 10
i=4 9
i=6 12
i=7 12

k	0	1	2	3	4	5	6	7
x	1	2	3	4	5	1	6	6